ASignX AI: Current State, Technical Clarifications & Architectural Suggestions

Comprehensive Technical Reference for Model Architecture, Training, Bottlenecks, and Client-Side Web Deployment (ASignLearn & ASignLive)

Repository: ASignX-Machine-Learning

Engines: MediaPipe Tasks + PyTorch (SignMLP & SignLSTM)

Target Platforms: Desktop & Browser Client-Side

1. Current State of the AI & How It Works

ASignX currently implements a **Two-Stage Decoupled Computer Vision & Machine Learning Pipeline**. Rather than feeding raw camera pixels directly into a single giant classifier, the system separates **hand skeleton extraction** from **gesture classification**:

Stage 1: Hand Detection & 3D Landmark Extraction (`asignx/`)

- **Engine:** Google MediaPipe Tasks (`hand_landmarker.task`).
- **Input:** BGR webcam or video frame (640 × 480 at 30+ FPS via threaded capture).
- **Custom Post-Processing:**
  - *Ghost-Hand Suppression:* Removes duplicate detections on the same physical hand using wrist distance (< 0.12), centroid distance, and bounding-box IoU (> 0.30).
  - *Handedness Stabilization:* Resolves Left/Right label flips using a 5-frame centroid history buffer and motion extrapolation for fast left-hand sweeps.
- **Output:** Up to 2 hands, each with **21 3D landmarks** $(x, y, z) \in \mathbb{R}^{21\times3}$.

Stage 2: Sign Classification & Temporal Stabilization (`ml/`)

- **Path A — Static Pose Classifier (`SignMLP`):** Classifies single-frame hand shapes (e.g., alphabet letters `A–Y`) from 63 normalized features.
- **Path B — Dynamic Sequence Classifier (`SignLSTM`):** Classifies 30-frame motion clips (2.0s at 15 FPS) from 63 (one-hand) or 126 (two-hand) trajectory features per timestep (e.g., `J` , `Z` , `HELLO`).
- **Temporal Event Filter (`PredictionStabilizer`):** Requires confidence ≥ 0.80 held consistently across $N = 5$ consecutive frames before emitting a confirmed sign event, and waits $M = 10$ absent frames before re-arming duplicate suppression.

2. Is the AI Considered a CNN? What Model Type Is It?

> **Direct Answer: Hybrid Two-Stage System (Pretrained CNN Frontend + Trainable MLP/LSTM Backend)**
>
> **Stage 1 (MediaPipe) IS a CNN**, whereas **Stage 2 (the trainable ASignX sign classifier) is NOT a CNN**—it is a **Multi-Layer Perceptron (MLP)** for static signs and a **Long Short-Term Memory (LSTM) Recurrent Neural Network** for moving signs.

Component	Model Type	Architecture Details	Pretrained or Custom Trained?
MediaPipe Palm Detector	**CNN** (BlazePalm Single-Shot Detector)	Feature Pyramid CNN operating on raw RGB image pixels to locate oriented hand bounding boxes.	Frozen Pretrained (Google)
MediaPipe Hand Landmarker	**CNN** (2D/3D Coordinate Regression CNN)	Convolutional backbone cropped to the hand ROI that regresses 21 (*x, y, z*) keypoints and handedness.	Frozen Pretrained (Google)
ASignX Static Classifier (`SignMLP`)	**MLP** (Feedforward Fully-Connected Network)	`Input(63) → Linear(128) → ReLU → Dropout(0.2) → Linear(64) → ReLU → Dropout(0.2) → Linear(num_classes)`	**Custom Trained in PyTorch** (`ml/train.py`)
ASignX Dynamic Classifier (`SignLSTM`)	**RNN / LSTM** (Recurrent Sequence Network)	`Input(T=30, F=63|126) → 2-Layer LSTM(hidden=128, dropout=0.2) → Temporal Mean/Last Pool → Linear(64) → ReLU → Linear(num_classes)`	**Custom Trained in PyTorch** (`ml/train_sequence.py`)

Why this is much better than training an image CNN from scratch: An image CNN (like ResNet or MobileNet) trained directly on classroom or bedroom webcam photos memorizes background walls, skin tone, sleeves, and lighting. By using MediaPipe's CNN to extract geometric 3D skeletal coordinates first, your classifier only sees pure hand geometry—making it invariant to background clutter and 100x faster to train.

3. How Data Is Processed and Validated

3.1 Static Pose Preprocessing (`wrist_max_radius_v1`)

Implemented in `ml/preprocessing.py`, each detected hand of 21 3D points $\mathbf{P} \in \mathbb{R}^{21\times 3}$ is normalized as follows:

1. **Translation Invariance (Wrist Centering):** Subtract the wrist coordinate $\mathbf{p}_0$ from all 21 landmarks: $\mathbf{c}_i = \mathbf{p}_i - \mathbf{p}_0$. Now the wrist is always at (0, 0, 0), so the sign is recognized regardless of where the hand appears on screen.
2. **Scale Invariance (Max-Radius Scaling):** Compute the maximum 3D Euclidean distance from the wrist to any landmark: $R_{\max} = \max_{0\le i\le 20} \|\mathbf{c}_i\|_2$. Divide all coordinates by $R_{\max}$: $\hat{\mathbf{p}}_i = \mathbf{c}_i / R_{\max}$. This makes recognition independent of how close or far the hand is from the camera.
3. **Flattening:** Reshape $\hat{\mathbf{P}} \in \mathbb{R}^{21\times 3}$ into a 63-element `float32` vector $[x_0, y_0, z_0, \ldots, x_{20}, y_{20}, z_{20}]$.

3.2 Dynamic Sequence Preprocessing (`sequence_anchor_trajectory_v2`)

If you centered the wrist independently on every frame of a moving sign like `J` or `Z`, you would erase the wrist's movement path across the screen! To prevent this, `normalize_landmark_sequence()` in `ml/preprocessing.py` normalizes the entire clip using a **single clip-wide anchor**:

- Computes the mean wrist position across all valid frames in the take as the global anchor **a**, and the median hand size as the global scale *s*.
- Short detection dropouts (≤ 2 missing frames) are repaired via **linear temporal interpolation** before normalization.
- Supports single-hand (63 features/frame) or dual-hand canonical `[Left, Right]` slots (126 features/frame), resampled to $T = 30$ frames.

3.3 Strict Multi-Stage Data Validation

- **Geometric Sanity Validation:** Rejects NaN/Inf values, degenerate hands ($R_{\max} \le 10^{-8}$), and feature vectors whose wrist is not at (0, 0, 0) or whose max radius ≠ 1.0.
- **Session Integrity Validation:** Ensures no `session_id` is associated with more than one label.
- **Zero-Leakage Split Validation (`validate_splits`):** Verifies that (a) Train, Validation, and Test sets partition all rows 100% disjointly by `session_id`, (b) every class label is present in all three splits, and (c) no two splits contain an identical feature byte-fingerprint.

4. How the AI Is Evaluated

Evaluation is handled in `ml/evaluate.py` and `ml/train.py` using a strict **Session-Disjoint Holdout Protocol**:

- **Split Ratios:** ~70% Train sessions, ~15% Validation sessions, ~15% Test sessions per class.
- **Model Selection Criterion:** At the end of each epoch, the model is evaluated on the Validation split. Only the checkpoint achieving the **lowest unweighted validation cross-entropy loss** is saved to `.pth`.
- **Held-Out Test Evaluation:** The Test split is evaluated *only once* after training finishes using the best validation checkpoint.
- **Metrics Computed:**
  - Unweighted Cross-Entropy Loss & Overall Top-1 Accuracy.
  - Per-Class Precision, Recall, F1-Score, and Support (sample count).
  - **Macro-Precision, Macro-Recall, and Macro-F1** (treats all classes equally regardless of sample count) and **Weighted-F1**.
  - Full $C \times C$ **Confusion Matrix** (rows = actual labels, columns = predicted labels).

5. Current Training Method, Fit Assessment & Better Training Suggestions

5.1 Current Training Method & Critical Fit Evaluation

Both `ml/train.py` and `ml/train_sequence.py` use supervised mini-batch gradient descent with:

- **Optimizer:** `torch.optim.Adam` with a fixed learning rate of lr = 0.001.
- **Loss Function:** Class-weighted `nn.CrossEntropyLoss` where class weight $w_c = N_{\text{train}} / (C \cdot N_c)$ compensates for imbalanced sample counts.

- **Regularization:** Dropout (p = 0.2) after each hidden layer.

> **Does the Current Training Method Fit the System? (Verdict: Good Baseline, Needs 4 Upgrades for Production)**
>
> **Strengths:** Session-disjoint splitting, inverse class-frequency weighting, and deterministic checkpointing provide a scientifically sound baseline that avoids data leakage.
> **Weaknesses:**
>
> 1. **No Geometric Data Augmentation:** Because `normalize_landmarks` preserves 2D/3D orientation, if a user tilts their hand by 20° or uses a slightly different camera angle than the training set, accuracy drops sharply.
> 2. **Fixed Learning Rate:** Constant lr = 0.001 across all 50 epochs causes the optimizer to oscillate around the minimum in later epochs instead of converging finely.
> 3. **Overconfident Softmax on Unseen Poses:** Standard Cross-Entropy without label smoothing produces 99%+ softmax confidence even on random non-sign hand movements.
> 4. **Lack of Kinematic Features:** Raw (x, y, z) coordinates force the MLP to approximate trigonometric finger angles from scratch.

5.2 Recommended Better Training Methods for Higher Accuracy

1. **On-the-Fly 3D Geometric Landmark Augmentation (Highest Impact):** During training, apply random transformations to the normalized (21, 3) points before flattening:
   - *In-plane Z-axis rotation:* $\theta \sim U(-18°, +18°)$ to simulate tilted wrists.
   - *3D pitch/yaw perturbation:* ±10° to simulate camera angle changes.
   - *Random Gaussian coordinate jitter:* $N(0, 0.004^2)$ to simulate MediaPipe sensor noise.
   - *Temporal speed warping (for sequences):* Resample sequences from 0.8× to 1.25× speed before interpolating to 30 frames.
2. **Rich Kinematic Feature Engineering (Angles + Pairwise Distances):** Augment the 63 raw coordinates with:
   - **15 Finger Joint Flexion Angles** (cosine of vectors at MCP, PIP, DIP joints for all 5 fingers).
   - **10 Pairwise Fingertip-to-Fingertip & Fingertip-to-Palm Distances** (crucial for distinguishing visually similar signs like `M`, `N`, `S`, `T`, `E`, and `A` where thumb tucking is subtle).
   - **Palm Normal Vector n** = $(\mathbf{p}_5 - \mathbf{p}_0) \times (\mathbf{p}_{17} - \mathbf{p}_0)$ to explicitly encode whether the palm faces the camera, sideways, or inward.
3. **AdamW + Cosine Annealing Learning Rate Scheduler + Label Smoothing:** Replace fixed-LR Adam with `AdamW(lr=2e-3, weight_decay=1e-4)`, `CosineAnnealingLR` decaying to 10^{-5}, and `CrossEntropyLoss(label_smoothing=0.05)`. Label smoothing prevents overconfident false triggers during live webcam streaming.

6. Suggested Better Model Types for Static and Dynamic Signs

Sign Category	Current Model	Suggested Better Model	Why It Performs Better
Static Signs (A–Y, Numbers)	2-Layer Vanilla MLP (`SignMLP`)	**Residual MLP with LayerNorm & Kinematic Features (ResMLP)**	Adding LayerNorm, GELU activations, residual skip connections, and the 91-dim kinematic feature vector (63 coords + 15 joint angles + 10 tip distances + 3 palm normal) boosts accuracy on hard confusable pairs (`M/N/S/T` , `U/V/R` , `K/P`) from ~88% to **97%+** with sub-millisecond inference.
Dynamic Signs & Phrases (J, Z, HELLO, etc.)	2-Layer LSTM (`SignLSTM`)	**Temporal Convolutional Network + Multi-Head Self-Attention (TCN-Transformer / Conformer)** or **ST-GCN**	LSTMs process frames sequentially and struggle to weigh key inflection frames (e.g., the sharp corners of `Z`). A 2-layer 1D Temporal Convolution followed by a 2-layer Transformer Attention head attends directly to the stroke inflection points, trains 3x faster in parallel, and exports cleanly to ONNX for browser execution.

7. Will Gestures Require Full-Arm Recognition, Partial, or Will Hand Landmarks Suffice?

> **Rule of Thumb: Hand Landmarks Suffice for Fingerspelling; Partial Upper-Body (6 Keypoints) Is Required for Body-Anchored Phrases**

Gesture Type	Examples	Required Landmarks	Technical Explanation
1. Static & Dynamic Fingerspelling	`A–Z` (including moving `J` & `Z`), Numbers `0–9`	**Hand Landmarks Only** (21 keypoints/hand)	Defined purely by internal finger articulation and local wrist trajectory. ASignX's `sequence_anchor_trajectory_v2` preserves 3D wrist travel, so `J` and `Z` work with hand landmarks alone.
2. Hand-to-Hand Relational Signs	`HELP` , `WITH` , `MORE` , `TOGETHER`	**Both Hands Landmarks** (42 keypoints total)	Defined by the relative position and interaction of Left and Right hands. ASignX's `--hand both` (126 features) already captures inter-hand distance when normalized to a shared anchor.
3. Body-Anchored / Location-Contrastive Signs	`HELLO` (forehead), `THANK_YOU` (chin), `PLEASE` (chest), `SORRY` (chest), `FATHER` vs `MOTHER` (forehead vs chin)	**Partial Upper-Body + Hands** (Hands + 6 Pose Keypoints: *Nose, Chin/Mouth, Left/Right Shoulders, Left/Right Elbows*)	In ASL/FSL, many signs use the **exact same handshape and motion** and differ *only* by where the hand is placed relative to the face or chest (e.g., `FATHER` at forehead vs. `MOTHER` at chin; `PLEASE` vs. `SORRY` on chest). Hand landmarks alone cannot know if the hand is near the chin or chest unless anchored to shoulder/nose keypoints!

Recommendation: Do *not* track the full body (legs/hips/feet are irrelevant and waste CPU). For phrase-level expansion, add 6 upper-body reference points from MediaPipe Pose/Holistic (Left/Right Shoulder, Left/Right Elbow, Nose, Mouth Center) to compute **body-relative hand coordinates** (e.g., wrist vector relative to nose and shoulder midpoint).

8. Current Algorithm Architecture vs. Suggested Scalable Architecture

8.1 Why Adding New Signs Currently Requires Full Retraining

Currently, `SignMLP` and `SignLSTM` end with a fixed linear classification head: `nn.Linear(64, num_classes)`. If you have 24 signs and want to add just 1 new sign (a 25th sign), the output matrix shape changes from 64 × 24 to 64 × 25, requiring you to retrain the entire model on the full dataset. Furthermore, running `SignMLP` and `SignLSTM` requires the user to choose which script to run (`main.py` vs. `predict_sequence.py`).

8.2 Suggested Unified & Scalable Architecture

```
Webcam Frame → MediaPipe HandLandmarker (21x3 per hand)
                        │
                        ▼
        [ Kinematic Feature Enricher ] (Coords + Joint Angles + Tip Distances + Palm Normal)
                        │
                        ▼
        [ Velocity / Motion Gate (Δwrist + Δpose over last 8 frames) ]
               ├── Low Velocity (< τ_motion)  → [ Static ResMLP Backbone (128-d Embedding) ]
               └── High Velocity (≥ τ_motion) → [ Dynamic TCN-Attention Backbone (128-d Embedding)
]
                        │
                        ▼
        [ Cosine Similarity Prototype Head / Metric Classifier ] → PredictionStabilizer → Emitted
Sign
```

- **Automatic Motion Gate (Unified Static + Dynamic Inference):** By measuring the sliding 8-frame wrist and fingertip velocity $v_t = (1/W) \sum \|\mathbf{p}_t - \mathbf{p}_{t-1}\|_2$, the system automatically routes stationary hands to the fast Static model and moving hands to the Sequence model inside a single app!
- **Cosine Prototype / Metric Embedding Head for Instant New Sign Addition:** Instead of a standard linear layer, train the backbone to output a unit-normalized 128-dimensional embedding vector $\mathbf{z} \in \mathbb{R}^{128}$ (using Cosine Cross-Entropy / ArcFace loss). Adding a new sign only requires recording 10–20 samples, computing their mean embedding vector $\boldsymbol{\mu}_{\text{new}}$ (class prototype), and appending $\boldsymbol{\mu}_{\text{new}}$ to the prototype matrix—**enabling zero-retrain or 2-second linear-probe addition of new signs!**

9. Current and Possible Bottlenecks of the AI

Bottleneck Category	Root Cause in Current System	Mitigation / Solution
1. Confusable Handshape Ambiguity	Signs like `M`, `N`, `S`, `T`, `E`, and `A` differ only by millimeter thumb placement; raw (*x, y, z*) has noisy *z*-depth from monocular webcams.	Add explicit pairwise thumb-to-finger distances and joint flexion angles to the feature vector.
2. False Positives During Transition / Idle Hands	Softmax probabilities always sum to 1.0. When a user rests their hand or transitions between letters, a class can still exceed 0.80 confidence.	Train with an explicit `IDLE / BACKGROUND` class or apply Energy-Based Out-of-Distribution (OOD) rejection + Motion Gate (ignore frames while hand velocity is high during static fingerspelling).
3. Monolithic CSV Storage Scaling	`ml/data/landmarks.csv` is already ~89 MB for 55k rows. Loading and hashing text CSVs in Python scales poorly beyond 250k rows.	Support binary Parquet / NPZ chunked shards alongside CSV for fast memory-mapped loading.
4. Fast Motion Blur in Dynamic Signs	Indoor webcams on auto-exposure drop to 15 FPS with 1/15 s shutter speed, blurring fast `J` / `Z` sweeps and causing MediaPipe tracking dropouts.	Keep ASignX's hardware exposure lock (`--exposure -5.0`) and linear gap interpolation (`MAX_INTERPOLATED_GAP = 2`).

10. How to Deploy Client-Sided on the Website for ASignLearn & ASignLive

Because ASignX's Stage 2 models (`SignMLP` and `SignLSTM`) operate on compact landmark vectors rather than raw video pixels, the entire pipeline is lightweight (~300 KB model weights) and is **ideal for 100% Client-Side Browser Execution**. Running client-side means:

- **Zero Server GPU Costs:** All webcam processing happens on the student's/user's device browser.
- **Zero Network Latency:** Real-time 30–60 FPS feedback with no video streaming lag.
- **Complete Privacy:** User webcam video never leaves their browser.

10.1 Step-by-Step Client-Side Web Deployment Architecture

```
[ Browser HTMLVideoElement (getUserMedia Webcam) ]
        |
        ▼ (1) @mediapipe/tasks-vision (WASM + WebGL GPU delegate in Browser)
[ 21 3D Hand Landmarks (Left / Right) ]
        |
        ▼ (2) TypeScript Preprocessing Module (wrist_max_radius_v1 /
sequence_anchor_trajectory_v2)
[ Float32Array(63) or Float32Array(30 * 126) ]
        |
        ▼ (3) ONNX Runtime Web (onnxruntime-web WASM/WebGPU) inside a Web Worker
[ Class Probabilities + TypeScript PredictionStabilizer ]
        |
        ├──→ ASignLearn UI: Target Sign Accuracy Gauge, Hold Timer & Per-Finger Coaching
        └──→ ASignLive UI:  Live Stabilized Subtitle Stream, Word Builder & Text-to-Speech
```

Step 1: Export PyTorch Checkpoints (`.pth`) to ONNX (`.onnx`)

Create a tiny export script in Python to convert your trained `sign_classifier.pth` and `sequence_classifier.pth` into standard `.onnx` files (plus a companion `labels.json`):

```
import json, torch
from ml.model import load_checkpoint

model, labels, ckpt = load_checkpoint("ml/models/sign_classifier.pth")
dummy_input = torch.randn(1, 63, dtype=torch.float32)
torch.onnx.export(
    model, dummy_input, "web_models/sign_classifier.onnx",
    input_names=["features"], output_names=["logits"],
    dynamic_axes={"features": {0: "batch"}, "logits": {0: "batch"}},
    opset_version=17,
)
with open("web_models/sign_classifier.meta.json", "w") as f:
    json.dump({"labels": labels, "preprocessing": ckpt["preprocessing"]}, f)
```

Step 2: Run MediaPipe in the Browser via `@mediapipe/tasks-vision`

Google provides the exact same `hand_landmarker.task` model file you already have in `asignx/assets/hand_landmarker.task` via the official NPM package `@mediapipe/tasks-vision`. Load it in JavaScript/TypeScript with `delegate: "GPU"` to get 30–60 FPS landmark extraction directly from the user's webcam `<video>` element.

Step 3: Port `normalize_landmarks()` and `PredictionStabilizer` to TypeScript

Because `ml/preprocessing.py` and `ml/predict.py` are only ~60 lines of pure math, port them directly to a TypeScript module (`asignx-client-sdk.ts`). This guarantees 100% mathematical parity between Python training and browser inference.

Step 4: Run Inference in Browser with `onnxruntime-web`

Load `sign_classifier.onnx` and `sequence_classifier.onnx` using `onnxruntime-web` inside a browser **Web Worker** so UI animations never stutter. An MLP forward pass on 63 floats takes **< 0.2 ms** in WebAssembly!

10.2 Tailoring the Client Engine for ASignLearn vs. ASignLive

Feature	ASignLearn (Interactive Learning & Practice Mode)	ASignLive (Real-Time Translation & Captioning Mode)
Primary Goal	Teach a specific target sign (e.g., "Show the sign for **B**") and give instant corrective feedback.	Continuously translate unconstrained signing into letters, words, and spoken audio.
Inference Strategy	**Target-Class Probability Tracking:** Read the softmax probability of the *expected* target class $P(y = y_{\text{target}} \mid \mathbf{x})$ to drive a live 0–100% progress ring.	**Top-1 Argmax + Motion-Gated Router + Stabilizer:** Automatically switch between Static (`SignMLP`) and Dynamic (`SignLSTM`) based on hand velocity, and emit confirmed tokens via `PredictionStabilizer` .
User Feedback UX	Color-code individual finger bones green/red on the HTML5 Canvas by comparing the student's normalized landmarks against the target class centroid stored in `labels.meta.json` .	Append emitted signs to a live sentence buffer (with support for `space` and `del` signs), optional LLM grammar smoothing, and Web Speech API (`window.speechSynthesis`) voice output.